

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут» ім. Ігоря Сікорського

Лабораторна робота №2
з дисципліни «Основи Проектування Трансляторів»

«РОЗРОБКА СИНТАКСИЧНОГО АНАЛІЗАТОРА»

Виконав студент групи: КВ-33

Козлов Сергій

Київ 2026

Мета роботи

Метою лабораторної роботи «Розробка синтаксичного аналізатора» є засвоєння теоретичного матеріалу та набуття практичного досвіду і практичних навичок розробки синтаксичних аналізаторів (парсерів).

Постановка задачі

1. Розробити програму синтаксичного аналізатора (СА) для підмножини мови програмування SIGNAL згідно граматики за варіантом.

2. Програма має забезпечувати наступне:

- читання рядка лексем та таблиць, згенерованих лексичним аналізатором, який було розроблено в лабораторній роботі «Розробка лексичного аналізатора»;
- синтаксичний аналіз (розбір) програми, поданої рядком лексем (алгоритм синтаксичного аналізатора вибирається за варіантом);
- побудову дерева розбору;
- формування таблиць ідентифікаторів та різних констант з повною інформацією, необхідною для генерування коду;
- формування лістингу вхідної програми з повідомленнями про лексичні та синтаксичні помилки.

Алгоритм синтаксичного аналізу

Варіант 6

Низхідний розбір за алгоритмом рекурсивного спуску.

Граматика

Варіант 6

Дана граматика не має лівої рекурсії і використовується без перетворень.

1. `<signal-program> --> <program>`
2. `<program> --> PROGRAM <procedure-identifier>;
<block>.`
3. `<block> --> <declarations> BEGIN <statements-list>
END`
4. `<statements-list> --> <empty>`
5. `<declarations> --> <constant-declarations>`
6. `<constant-declarations> --> CONST
<constant-declarations-list> | <empty>`
7. `<constant-declarations-list> -->
<constant-declaration> <constant-declarations-list> |
<empty>`
8. `<constant-declaration> --> <constant-identifier> =
<constant>;`
9. `<constant> --> '<complex-number>'`
10. `<complex-number> --> <left-part> <right-part>`
11. `<left-part> --> <unsigned-integer> | <empty>`
12. `<right-part> --> ,<unsigned-integer> | $EXP(
<unsigned-integer>) | <empty>`
13. `<constant-identifier> --> <identifier>`
14. `<procedure-identifier> --> <identifier>`
15. `<identifier> --> <letter><string>`
16. `<string> --> <letter><string> | <digit><string> |
<empty>`
17. `<unsigned-integer> --> <digit><digits-string>`
18. `<digits-string> --> <digit><digits-string> |
<empty>`
19. `<digit> --> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9`
20. `<letter> --> A | B | C | D | ... | Z`

Обробка помилок і відновлення після помилок

Для розробленого СА були додані процедури для обробки помилок (error handling) та відновлення після помилок (error recovery). Це дозволяє СА продовжувати роботу після віднайдення синтаксичної помилки, що дозволяє знаходити декілька синтаксичних помилок за один прохід.

Даний СА використовує 2 стратегії:

- Відновлення після паніки (Panic mode recovery, PMR);
- Автоматична підстановка крапок з комою (Automatic semicolon insertion, ASI);

При віднайденні помилки (паніці) PMR дозволяє пропустити блок помилкових токенів, і відновити роботу з наступної безпечної позиції: певного ключового слова чи обмежувача ‘;’.

ASI дозволяє підставляти відсутні ‘;’ після визначень констант і продовжувати аналіз. Без цього механізму, PMR при відсутній ‘;’ пропускати і наступну константу, якщо така є, що потенційно приховуватиме помилки.

Дерева розбору тестових програм

Наведемо декілька прикладів дерев розбору для заданої граматики.

Аналогічний розбір для кожного тестового файлу наведений в Додатку 2.

Файл: valid-empty.signal

```
PROGRAM EMPTY;  
BEGIN  
END.
```

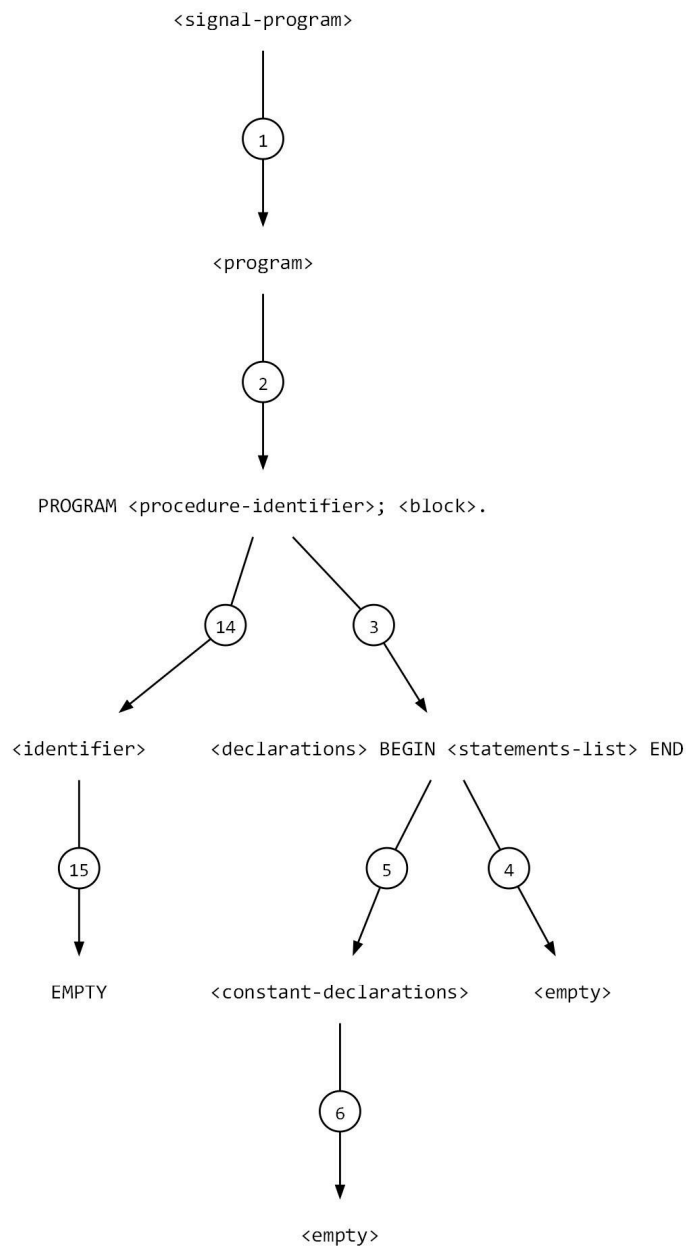


Рис. 1 – Дерево розбору для файлу `valid-empty.signal`

Файл: valid-basic.signal

```
PROGRAM HELLO;
CONST
  X = '42';
BEGIN
END.
```

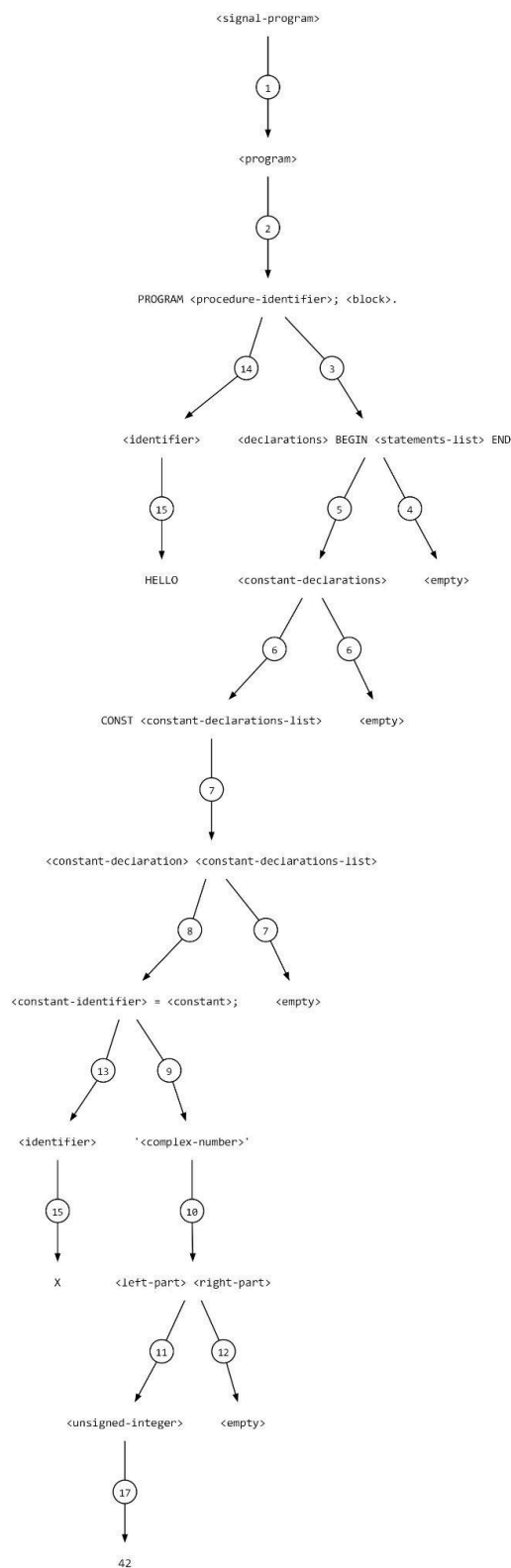


Рис. 2 – Дерево разбору для файла valid-basic.signal

Додатки

Додаток 1. Лістинг коду програми

Додаток 2. Результати тестування

Github

<https://github.com/Teollan/2026-opt-labs>